JavaScript is disabled on your browser.

**Java SE 27 & JDK 27**

# Class AccessibleObject

java.lang.Object
    java.lang.reflect.AccessibleObject

**All Implemented Interfaces:**

`AnnotatedElement`

**Direct Known Subclasses:**

`Executable`, `Field`

---

```
public class AccessibleObject
extends Object
implements AnnotatedElement
```

The `AccessibleObject` class is the base class for `Field`, `Method`, and `Constructor` objects (known as *reflected objects*). It provides the ability to flag a reflected object as suppressing checks for Java language access control when it is used. This permits sophisticated applications with sufficient privilege, such as Java Object Serialization or other persistence mechanisms, to manipulate objects in a manner that would normally be prohibited.

Java language access control prevents use of private members outside their top-level class; package access members outside their package; protected members outside their package or subclasses; and public members outside their module unless they are declared in an `exported` package and the user `reads` their module. By default, Java language access control is enforced (with one variation) when `Fields`, `Methods`, or `Constructors` are used to get or set fields, to invoke methods, or to create and initialize new instances of classes, respectively. Every reflected object checks that the code using it is in an appropriate class, package, or module. The check when invoked by JNI code with no Java class on the stack only succeeds if the member and the declaring class are public, and the class is in a package that is exported to all modules.

The one variation from Java language access control is that the checks by reflected objects assume readability. That is, the module containing the use of a reflected object is assumed to read the module in which the underlying field, method, or constructor is declared.

Whether the checks for Java language access control can be suppressed (and thus, whether access can be enabled) depends on whether the reflected object corresponds to a member in an exported or open package (see `setAccessible(boolean)`).

**See *Java Language Specification*:**

6.6 Access Control

**Since:**

1.2

**External Specifications**

Java Native Interface Specification

## Constructor Summary

**Constructors**

| Modifier | Constructor | Description |
|---|---|---|
| protected | `AccessibleObject()` | **Deprecated.** |

## Method Summary

**All Methods** | **Static Methods**

| Modifier and Type | Method | Description |
|---|---|---|
| final boolean | `canAccess(Object obj)` | Test if the caller can access this reflected object. |
| <T extends Annotation> T | `getAnnotation (Class<T> annotationClas` | Returns this element's annotation for the specified type if such an annotation is *present*, else null. |
| Annotation[] | `getAnnotations()` | Returns annotations that are *present* on this element. |
| <T extends Annotation> T[] | `getAnnotationsByType (Class<T> annotationClas` | Returns annotations that are *associated* with this element. |
| <T extends Annotation> T | `getDeclaredAnnotation (Class<T> annotationClas` | Returns this element's annotation for the specified type if such an annotation is *directly present*, else null. |
| Annotation[] | `getDeclaredAnnotations()` | Returns annotations that are *directly present* on this element. |

| | | |
|---|---|---|
| <T extends Annotation> T[] | **getDeclaredAnnotationsBy**(Class<T> annotationClas | Returns this element's annotation(s) for the specified type if such annotations are either *directly present* or *indirectly present*. |
| boolean | **isAccessible**() | **Deprecated.** This method is deprecated because its name hints that it checks if the reflected object is accessible when it actually indicates if the checks for Java language access control are suppressed. |
| boolean | **isAnnotationPresent**(Class<? extends Annotation> annotationCl | Returns true if an annotation for the specified type is *present* on this element, else false. |
| void | **setAccessible**(boolean flag) | Set the `accessible` flag for this reflected object to the indicated boolean value. |
| static void | **setAccessible**(AccessibleObject[] arra boolean flag) | Convenience method to set the `accessible` flag for an array of reflected objects. |
| final boolean | **trySetAccessible**() | Set the `accessible` flag for this reflected object to `true` if possible. |

**Methods declared in class Object**

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

## Constructor Details

### AccessibleObject

```
@Deprecated(since="17")
protected AccessibleObject()
```

**Deprecated.**

Constructor: only used by the Java Virtual Machine.

## Method Details

### setAccessible

```
public static void setAccessible(AccessibleObject[] array,
                                 boolean flag)
```

Convenience method to set the `accessible` flag for an array of reflected objects.

This method may be used to enable access to all reflected objects in the array when access to each reflected object can be enabled as specified by `setAccessible(boolean)`.

A `SecurityException` is thrown if any of the elements of the input `array` is a `Constructor` object for the class `java.lang.Class` and `flag` is true.

**Parameters:**

`array` - the array of AccessibleObjects

`flag` - the new value for the `accessible` flag in each object

**Throws:**

`InaccessibleObjectException` - if access cannot be enabled for all objects in the array

`SecurityException` - if an element in the array is a constructor for `java.lang.Class`

`NullPointerException` - if `array` or any of its elements is `null`

## setAccessible

```
public void setAccessible(boolean flag)
```

Set the `accessible` flag for this reflected object to the indicated boolean value. A value of `true` indicates that the reflected object should suppress checks for Java language access control when it is used. A value of `false` indicates that the reflected object should enforce checks for Java language access control when it is used, with the variation noted in the class description.

This method may be used by a caller in class `C` to enable access to a `member` of `declaring class` `D` if any of the following hold:

- `C` and `D` are in the same module.
- The member is `public` and `D` is `public` in a package that the module containing `D` `exports` to at least the module containing `C`.
- The member is `protected static`, `D` is `public` in a package that the module containing `D` exports to at least the module containing `C`, and `C` is a subclass of `D`.
- `D` is in a package that the module containing `D` `opens` to at least the module containing `C`. All packages in unnamed and open modules are open to all modules and so this method always succeeds when `D` is in an unnamed or open module.

This method may be used by JNI code with no caller class on the stack to enable access to a `member` of `declaring class` `D` if and only if:

- The member is `public` and `D` is `public` in a package that the module containing `D` `exports` unconditionally.

This method cannot be used to enable access to private members, members with default (package) access, protected instance members, or protected constructors when the declaring class is in a different module to the caller and the package containing the declaring class is not open to the caller's module.

**Parameters:**

`flag` - the new value for the `accessible` flag

**Throws:**

`InaccessibleObjectException` - if access cannot be enabled

**External Specifications**

Java Native Interface Specification

**See Also:**

`trySetAccessible()`,

MethodHandles.privateLookupIn(Class, MethodHandles.Lookup)

## trySetAccessible

```
public final boolean trySetAccessible()
```

Set the `accessible` flag for this reflected object to `true` if possible. This method sets the `accessible` flag, as if by invoking `setAccessible(true)`, and returns the possibly-updated value for the `accessible` flag. If access cannot be enabled, i.e. the checks or Java language access control cannot be suppressed, this method returns `false` (as opposed to `setAccessible(true)` throwing `InaccessibleObjectException` when it fails).

This method is a no-op if the `accessible` flag for this reflected object is `true`.

For example, a caller can invoke `trySetAccessible` on a `Method` object for a private instance method `p.T::privateMethod` to suppress the checks for Java language access control when the `Method` is invoked. If `p.T` class is in a different module to the caller and package `p` is open to at least the caller's module, the code below successfully sets the `accessible` flag to `true`.

```
p.T obj = ....;  // instance of p.T
:
Method m = p.T.class.getDeclaredMethod("privateMethod");
if (m.trySetAccessible()) {
    m.invoke(obj);
} else {
    // package p is not opened to the caller to access private member of T
    ...
}
```

If this method is invoked by JNI code with no caller class on the stack, the `accessible` flag can only be set if the member and the declaring class are public, and the class is in a package that is exported unconditionally.

**Returns:**

`true` if the `accessible` flag is set to `true`; `false` if access cannot be enabled.

**Since:**

9

**External Specifications**

Java Native Interface Specification

**See Also:**

MethodHandles.privateLookupIn(Class, MethodHandles.Lookup)

## isAccessible

```
@Deprecated(since="9")
public boolean isAccessible()
```

**Deprecated.**
*This method is deprecated because its name hints that it checks if the reflected object is accessible when it actually indicates if the checks for Java language access control are suppressed. This method may return `false` on a reflected object that is accessible to the caller. To test if this reflected object is accessible, it should use `canAccess(Object)`.*

Get the value of the `accessible` flag for this reflected object.

**Returns:**

the value of the object's `accessible` flag

## canAccess

```
public final boolean canAccess(Object obj)
```

Test if the caller can access this reflected object. If this reflected object corresponds to an instance method or field then this method tests if the caller can access the given `obj` with the reflected object. For instance methods or fields then the `obj` argument must be an instance of the `declaring class`. For static members and constructors then `obj` must be `null`.

This method returns `true` if the `accessible` flag is set to `true`, i.e. the checks for Java language access control are suppressed, or if the caller can access the member as specified in *The Java Language Specification*, with the variation noted in the class description. If this method is invoked by JNI code with no caller class on the stack, this method returns `true` if the member and the declaring class are public, and the class is in a package that is exported unconditionally.

**Parameters:**

`obj` - an instance object of the declaring class of this reflected object if it is an instance method or field

**Returns:**

`true` if the caller can access this reflected object.

**Throws:**

`IllegalArgumentException` -

- if this reflected object is a static member or constructor and the given `obj` is non-`null`, or

- if this reflected object is an instance method or field and the given `obj` is `null` or of type that is not a subclass of the `declaring class` of the member.

**See *Java Language Specification*:**

6.6 Access Control

**Since:**

9

**External Specifications**

Java Native Interface Specification

**See Also:**

`trySetAccessible()`, `setAccessible(boolean)`

## getAnnotation

```
public <T extends Annotation> T getAnnotation
(Class<T> annotationClass)
```

Returns this element's annotation for the specified type if such an annotation is *present*, else null.

Note that any annotation returned by this method is a declaration annotation.

**Specified by:**

`getAnnotation` in interface `AnnotatedElement`

**Implementation Requirements:**

The default implementation throws `UnsupportedOperationException`; subclasses should override this method.

**Type Parameters:**

T - the type of the annotation to query for and return if present

**Parameters:**

`annotationClass` - the Class object corresponding to the annotation type

**Returns:**

this element's annotation for the specified annotation type if present on this element, else null

**Throws:**

`NullPointerException` - if the given annotation class is null

**Since:**

1.5

## isAnnotationPresent

```
public boolean isAnnotationPresent
(Class<? extends Annotation> annotationClass)
```

Returns true if an annotation for the specified type is *present* on this element, else false. This method is designed primarily for convenient access to marker annotations.

The truth value returned by this method is equivalent to:
`getAnnotation(annotationClass) != null`

**Specified by:**

`isAnnotationPresent` in interface `AnnotatedElement`

**Parameters:**

`annotationClass` - the Class object corresponding to the annotation type

**Returns:**

true if an annotation for the specified annotation type is present on this element, else false

**Throws:**

`NullPointerException` - if the given annotation class is null

**Since:**

1.5

## getAnnotationsByType

```
public <T extends Annotation> T[] getAnnotationsByType
(Class<T> annotationClass)
```

Returns annotations that are *associated* with this element. If there are no annotations *associated* with this element, the return value is an array of length 0. The difference between this method and `AnnotatedElement.getAnnotation(Class)` is that this method detects if its argument is a *repeatable annotation type* (JLS §9.6), and if so, attempts to find one or more annotations of that type by "looking through" a container annotation. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Note that any annotations returned by this method are declaration annotations.

**Specified by:**

`getAnnotationsByType` in interface `AnnotatedElement`

**Implementation Requirements:**

The default implementation throws `UnsupportedOperationException`; subclasses should override this method.

**Type Parameters:**

`T` - the type of the annotation to query for and return if present

**Parameters:**

`annotationClass` - the Class object corresponding to the annotation type

**Returns:**

all this element's annotations for the specified annotation type if associated with this element, else an array of length zero

**Throws:**

`NullPointerException` - if the given annotation class is null

**Since:**

1.8

## getAnnotations

```
public Annotation[] getAnnotations()
```

Returns annotations that are *present* on this element. If there are no annotations *present* on this element, the return value is an array of length 0. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Note that any annotations returned by this method are declaration annotations.

**Specified by:**

`getAnnotations` in interface `AnnotatedElement`

**Returns:**

annotations present on this element

**Since:**

1.5

## getDeclaredAnnotation

```
public <T extends Annotation> T getDeclaredAnnotation
(Class<T> annotationClass)
```

Returns this element's annotation for the specified type if such an annotation is *directly present*, else null. This method ignores inherited annotations. (Returns null if no annotations are directly present on this element.)

Note that any annotation returned by this method is a declaration annotation.

**Specified by:**

`getDeclaredAnnotation` in interface `AnnotatedElement`

**Type Parameters:**

T - the type of the annotation to query for and return if directly present

**Parameters:**

`annotationClass` - the Class object corresponding to the annotation type

**Returns:**

this element's annotation for the specified annotation type if directly present on this element, else null

**Throws:**

`NullPointerException` - if the given annotation class is null

**Since:**

1.8

## getDeclaredAnnotationsByType

```
public <T extends Annotation> T[] getDeclaredAnnotationsByType
(Class<T> annotationClass)
```

Returns this element's annotation(s) for the specified type if such annotations are either *directly present* or *indirectly present*. This method ignores inherited annotations. If there are no specified annotations directly or indirectly present on this element, the return value is an array of length 0. The difference between this method and `AnnotatedElement.getDeclaredAnnotation(Class)` is that this method detects if its argument is a *repeatable annotation type* (JLS §9.6), and if so, attempts to find one or more annotations of that type by "looking through" a container annotation if one is present. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Note that any annotations returned by this method are declaration annotations.

**Specified by:**

`getDeclaredAnnotationsByType` in interface `AnnotatedElement`

**Type Parameters:**

T - the type of the annotation to query for and return if directly or indirectly present

**Parameters:**

`annotationClass` - the Class object corresponding to the annotation type

**Returns:**

all this element's annotations for the specified annotation type if directly or indirectly present on this element, else an array of length zero

**Throws:**

`NullPointerException` - if the given annotation class is null

**Since:**

1.8

## getDeclaredAnnotations

```
public Annotation[] getDeclaredAnnotations()
```

Returns annotations that are *directly present* on this element. This method ignores inherited annotations. If there are no annotations *directly present* on this element, the return value is an array of length 0. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Note that any annotations returned by this method are declaration annotations.

**Specified by:**

`getDeclaredAnnotations` in interface `AnnotatedElement`

**Implementation Requirements:**

The default implementation throws `UnsupportedOperationException`; subclasses should override this method.

**Returns:**

annotations directly present on this element

**Since:**

1.5

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.